ECA1418 - Embedded Systems Assignment

Particular	Details
Department:	CSE-AI
Programme:	B.TECH
Student Name	M.Nandhini
Student Register Number	192472318
Course Code & Course Name	ECA1418-Embedded System
Academic Year / Batch	2026 / 2024-2028
Faculty Name	Dr. H Shree Kumar
Assignment Title	Design, Develop, Debug and Evaluate a Real-Time Smart Home Automation and Security System Using RTOS, Embedded C and SoC-Based Edge Processing
Date of Issue	26/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) — CO	● CO1: Analyze and complex and networked embedded systems using formal design methodologies, embedded architectures, and communication protocols (such as CAN and CAN FD), for real-world applications. ● CO3: Analyze RTOS-based task scheduling, synchronization, and communication mechanisms to achieve reliable embedded system performance. ● CO4: Design and optimize embedded systems using RTOS-based scheduling, memory management, interrupt handling, and resource-sharing mechanisms to achieve efficient performance in real-time applications. ● CO5: Design and develop AI-enabled embedded systems using modern SoC platforms with effective communication, debugging, and version control techniques for real-time applications.
Bloom's Taxonomy Level	L5 – Evaluate; L6 – Create
SDG Mapping	SDG 7 – Affordable and Clean Energy; SDG 11 – Sustainable Cities and Communities

Design, develop, debug, and evaluate a real-time embedded home-automation and security system for a multi-room residential unit consisting of one central automation hub and four to six room-level sensor/actuator nodes controlling lighting, HVAC, and door/window security. The proposed system shall use a finite-state-machine or dataflow model to formalize system behaviour, RTOS-based task scheduling for sensor sampling, actuator control, and security-

event handling, interrupt-driven event detection (motion, door/window contact), a PID or ON-OFF control algorithm for HVAC regulation, and shall include a minimum of one MCU/SoC platform selection with justification, an RTOS task set with priority assignment and schedulability analysis (RMS/EDF), semaphore/mutex-based shared-resource protection, watchdog-based fault recovery, and a remote-notification mechanism (e.g., MQTT/HTTP to a mobile or web dashboard) for security alerts. Students shall debug at least one inter-module communication link using a logic analyzer and/or JTAG, and shall validate the proposed system using suitable simulation/emulation and firmware test-automation tools by conducting at least four controlled fault or security-event scenarios (e.g., sensor failure, unauthorized access attempt, communication loss, power interruption) and evaluating the system using a minimum of six measurable parameters, such as response latency, false-alarm rate, task deadline miss rate, power consumption, actuation accuracy, and system uptime. Students shall analyze the results, assess the real-time responsiveness, reliability, energy efficiency, and safety of the proposed system, and justify the major engineering decisions and technology choices made.

Detailed Assessment Rubric with Performance Levels (Mapped to Assignment Requirements)

S.N	Assessment Criterion	Marks	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
1	Problem Understanding & Requirements Formulation	10	Clearly identifies all requirements: multi-room sensing/actuation, security event handling, real-time response deadlines, connectivity, and safety/energy-efficiency needs.	Identifies most requirements with minor omissions.	Identifies basic requirements but lacks completeness.	Requirements poorly understood or incomplete.
2	Application of Embedded Systems, RTOS & Control Knowledge	20	Correctly applies Embedded C, FSM/state-machine modeling, RTOS task/priority concepts, semaphores/queues, interrupt/timer handling, and PID/ON-OFF control concepts.	Applies most concepts correctly with minor technical errors.	Partial implementation of required embedded, control, and RTOS concepts.	Major gaps in embedded systems, control, and RTOS concepts.
3	Solution Design, Implementation & Integration	20	Fully functional multi-node home-automation system integrating RTOS scheduling, sensor/actuator control, security event detection, remote notification, and watchdog-based fail-safe. All components operate cohesively.	Most required components implemented and integrated with minor issues.	Basic implementation completed but lacks full integration or functionality.	Incomplete implementation with major design/configuration flaws.
4	Use of Appropriate Modern Tools & Debugging Techniques	10	Effectively uses simulation/hardware tools (e.g., Proteus/Wokwi, PlatformIO/Arduino IDE, Python/PySerial test scripts) and debugs the system using a logic analyzer and/or JTAG, correctly interpreting captured traces.	Appropriate tools used correctly with adequate evidence.	Limited use of tools or insufficient utilization of outputs.	Minimal or inappropriate use of engineering/debugging tools.
5	Results, Testing & Validation	15	Conducts ≥4 fault/event scenarios (e.g., sensor failure, unauthorized	Required testing completed with	Limited testing, fewer scenarios, or	Insufficient testing and validation.

			access attempt, communication loss, power interruption) and evaluates ≥6 metrics (response latency, false-alarm rate, task deadline miss rate, power consumption, actuation accuracy, system uptime) with graphs, logs, and quantitative evidence.	adequate metric collection and analysis.	incomplete metric evaluation.	
6	Analysis, Trade-offs & Engineering Justification	15	Thoroughly evaluates real-time responsiveness, reliability, energy efficiency, scalability, and control-algorithm performance. Design decisions justified using data and results.	Sound analysis with reasonable justification of design choices.	Limited analysis and weak justification.	Conclusions unsupported by evidence.
7	Broader Considerations & Professional Responsibility	5	Demonstrates strong consideration of occupant safety, data/privacy protection, secure firmware practices, and energy sustainability.	Addresses most broader considerations adequately.	Limited discussion of professional or ethical implications.	Negligible consideration of broader impacts.
8	Technical Documentation & Reflection	5	Professional report and a thoughtful 300–500 word reflection addressing all required questions with clear evidence of learning, problem-solving and critical self-assessment.	Complete report and reflection addressing most questions with good insight.	Reflection addresses only some questions or lacks depth; documentation partially complete.	Poor documentation and/or missing, superficial, or incomplete reflection.
	TOTAL	**100**				

Submission Requirements:

Students shall submit the following deliverables as part of the assignment evaluation:

S.No.	Deliverable	Description
1	**Technical Report**	Comprehensive report (recommended 15–25 pages) covering requirements analysis, system architecture, RTOS/control design, implementation methodology, testing procedures, results, analysis, engineering decisions, and conclusions.
2	**System/Node Architecture Diagram**	Detailed block diagram showing the home-automation hub, sensor/actuator nodes (lighting, HVAC, door/window security), MCU/SoC platform, connectivity links, and fail-safe/watchdog mechanisms.
3	**State-Machine / Task Scheduling Design Document**	FSM/Mealy-Moore or dataflow model of system behaviour, RTOS task table with priorities and periods, interrupt/timer design, and synchronization (semaphore/mutex) strategy.

4	**Firmware Source Code / Configuration Files**	Embedded C source and header files, control-algorithm (PID/ON-OFF) implementation, build/toolchain configuration, and version-control (Git) history.
5	**Simulation/Emulation Project Files**	Original project files generated using Proteus, Wokwi, Arduino IDE/PlatformIO, or equivalent simulation/emulation platforms.
6	**Debugging Evidence (Logic Analyzer / JTAG)**	Captured bus/protocol traces (e.g., UART, SPI, I²C) and/or JTAG session logs used to identify and resolve at least one communication or firmware fault.
7	**Fault/Event Testing Evidence**	Documentation and screenshots/logs of at least four controlled fault or security-event scenarios, including test methodology, detection mechanism, mitigation action, and observations.
8	**Performance Evaluation Results**	Collected data, graphs, and tables for at least six evaluation metrics such as response latency, false-alarm rate, deadline miss rate, power consumption, actuation accuracy, and system uptime.
9	**Engineering Analysis & Design Justification**	Discussion of real-time responsiveness, reliability, energy efficiency, scalability, and justification of key technology and design choices (e.g., SoC platform, RTOS scheduling policy, control algorithm).
10	**Individual Reflection (300–500 words per student)**	Each student must submit an individual reflection addressing: (i) most challenging aspect of the problem, (ii) most useful embedded/RTOS concept, (iii) most useful control/debugging concept, (iv) technical problem encountered, (v) how it was solved, (vi) lessons learned from modern tools, (vii) engineering decision they would change, and (viii) how the assignment improved their understanding of real-world embedded systems.

1. INTRODUCTION

Smart home automation is an embedded system application that helps improve household comfort, security, and energy efficiency. A smart home can use sensors to monitor room conditions and control devices such as lights, HVAC systems, and doors or windows.This project focuses on designing a real-time smart home automation and security system using Embedded C, RTOS, and SoC-based edge processing. The system consists of a central automation hub and multiple room-level sensor and actuator nodes. The RTOS manages different tasks such as sensor monitoring, actuator control, and security-event handling.The system also uses interrupt-based event detection, HVAC control, synchronization mechanisms, watchdog-based fault recovery, and remote notification. The proposed system is tested using different fault and security scenarios and evaluated using parameters such as response latency, false-alarm rate, power consumption, actuation accuracy, and system uptime. This helps determine the real-time performance, reliability, safety, and energy efficiency of the system.

2. PROBLEM STATEMENT

In a multi-room home, different devices such as lights, HVAC systems, and security sensors need to be monitored and controlled at the same time. A normal system may not respond quickly to important events such as motion detection, unauthorized entry, sensor failure, or communication loss. Therefore, this project aims to develop a real-time smart home automation and security system using Embedded C and RTOS. The system should monitor sensors, control actuators, detect security events, manage tasks according to their priorities, and provide remote alerts. It should also include proper synchronization, watchdog-based fault recovery, and HVAC control. The system will be tested under different fault and security conditions and its performance will be evaluated using measurable parameters such as response latency, false-alarm rate, power consumption, actuation accuracy, task deadline misses, and system uptime.

3. OBJECTIVES

The main objectives of the proposed system are:

1. To design a real-time smart home automation and security system for multiple rooms.
2. To monitor room conditions and control lighting, HVAC, and door/window security devices.
3. To implement RTOS-based task scheduling for sensor monitoring, actuator control, and security events.
4. To use interrupt-based detection for motion and door/window security events.
5. To implement ON-OFF or PID-based control for HVAC temperature regulation.
6. To use semaphore or mutex mechanisms for safe sharing of system resources.

7. To implement watchdog-based fault recovery to improve system reliability.
8. To provide remote security notifications using MQTT or HTTP communication.
9. To test the system using different fault and security-event scenarios.
10. To evaluate the system using response latency, false-alarm rate, deadline miss rate, power consumption, actuation accuracy, and system uptime.

4. SYSTEM REQUIREMENTS

The proposed smart home automation and security system requires both hardware and software components to perform real-time monitoring and control.

4.1 Hardware Requirements

1. MCU/SoC: Acts as the main processing unit for the automation system.
2. Motion Sensor: Detects movement inside the rooms.
3. Door/Window Sensor: Detects unauthorized opening of doors or windows.
4. Temperature Sensor: Measures the room temperature for HVAC control.
5. Light/Lighting Module: Used for automatic lighting control.
6. HVAC Actuator: Controls heating or cooling according to the required temperature.
7. Communication Interface: Provides communication between the hub and room nodes.
8. Watchdog Timer: Helps recover the system during software or hardware faults.
9. Power Supply: Provides the required power to the controller, sensors, and actuators.

4.2 Software Requirements

1. Embedded C programming environment.
2. RTOS for task scheduling and management.
3. Simulation/emulation software such as Proteus, Wokwi, Arduino IDE, or equivalent.
4. Communication protocol such as MQTT or HTTP for remote notification.
5. Debugging tools such as a logic analyzer or JTAG.
6. Version-control system such as Git for maintaining firmware source code.

4.3 Functional Requirements

- The system shall monitor sensors continuously.
- The system shall control lighting and HVAC devices.
- The system shall detect motion and door/window security events.
- Security events shall generate appropriate alerts.
- RTOS tasks shall be assigned suitable priorities.
- Shared resources shall be protected using semaphore or mutex mechanisms.
- The watchdog shall provide fault recovery.

- The system shall support testing under fault and security-event conditions.

4.4 Performance Requirements

The system shall be evaluated using measurable parameters including:

- Response latency
- False-alarm rate
- Task deadline miss rate
- Power consumption
- Actuation accuracy
- System uptime

5. PROPOSED SYSTEM

The proposed system is a real-time smart home automation and security system designed for a multi-room residential environment. It consists of one central automation hub and four to six room-level sensor and actuator nodes.

The room nodes collect information from sensors such as temperature, motion, and door/window sensors. This information is sent to the central hub for processing. Based on the sensor inputs, the system controls devices such as lights and HVAC units.

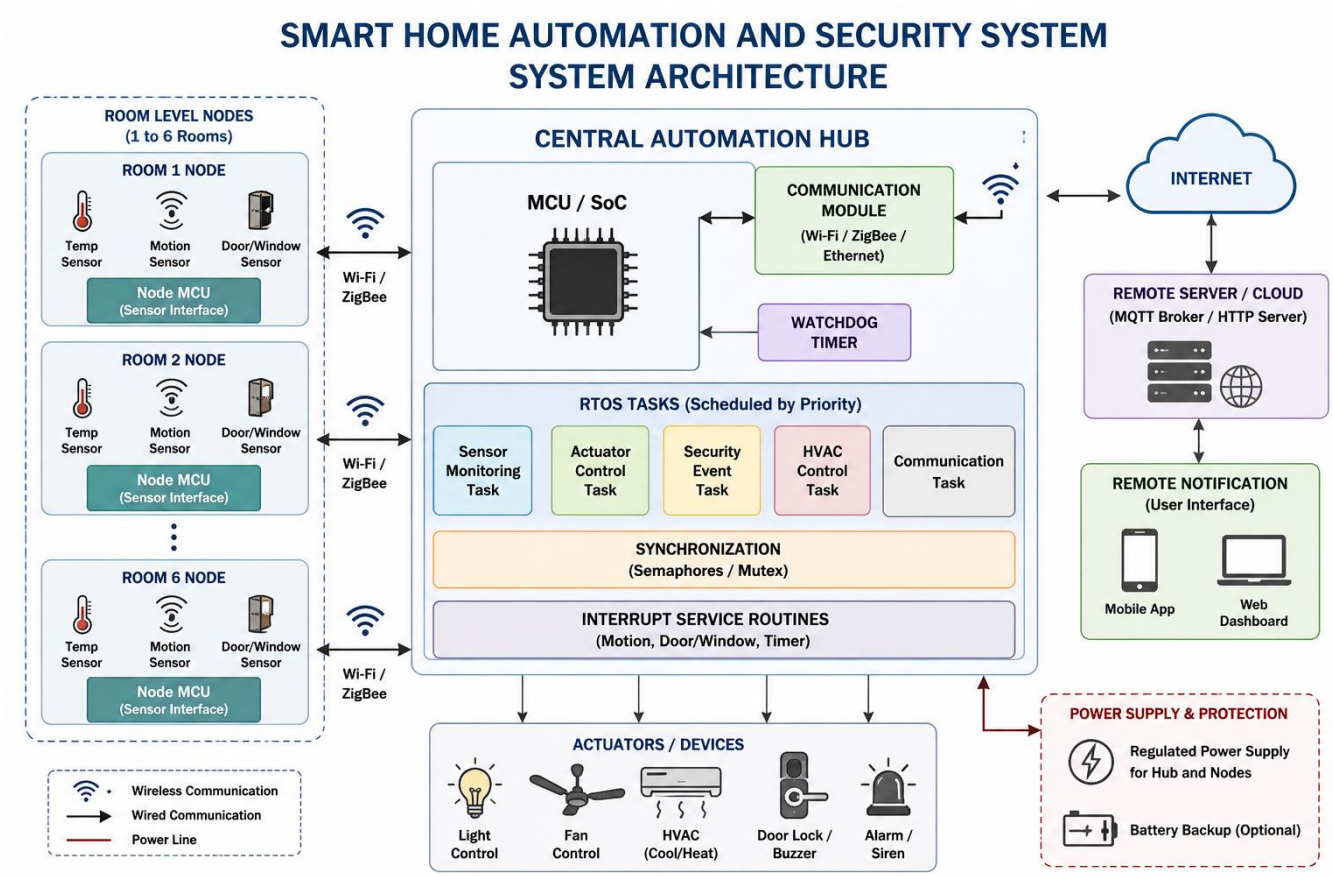
An RTOS is used in the central controller to manage different tasks according to their priorities and timing requirements. Sensor monitoring, actuator control, and security-event handling are implemented as separate tasks. Interrupts are used for important events such as motion detection and unauthorized door or window opening.

For HVAC regulation, an ON-OFF or PID control method can be used to maintain the required room temperature. Semaphores or mutexes are used when multiple tasks access shared resources. A watchdog mechanism is included to recover the system when an unexpected software or hardware fault occurs.

When a security event is detected, the system can send a remote notification through MQTT or HTTP. The overall system is validated using simulation/emulation and controlled fault scenarios to evaluate its response, reliability, safety, and energy efficiency.

6. SYSTEM ARCHITECTURE

The proposed system follows a central hub and multi-node architecture. A central automation hub manages the sensor and actuator nodes installed in different rooms. Each room node collects sensor information and communicates it to the central hub.



7. MCU/SoC PLATFORM SELECTION AND JUSTIFICATION

For the proposed smart home automation system, an **ESP32-based MCU/SoC platform** is selected as the main controller. ESP32 is suitable for this application because it provides sufficient processing capability, built-in wireless connectivity, GPIO interfaces, timers, and interrupt support in a single platform.

The ESP32 can interface with different sensors and actuators used in the room nodes. Its wireless connectivity can be used for communication between the central hub, room nodes, and remote notification system. This reduces the need for a separate communication controller.

The platform also supports the implementation of RTOS-based tasks. Different tasks can be assigned for sensor monitoring, actuator control, security-event processing, HVAC control, and communication. Interrupts can be used for time-critical events such as motion detection and door/window status changes.

7.1 Reasons for Selecting ESP32

- Supports Embedded C programming.
- Provides built-in Wi-Fi and Bluetooth connectivity.
- Provides multiple GPIO pins for sensors and actuators.

- Supports interrupts and timer-based operations.
- Suitable for RTOS-based task management.
- Provides adequate processing capability for edge-based operations.
- Can be integrated with MQTT/HTTP-based communication.

Therefore, the ESP32 provides a practical balance between processing capability, connectivity, real-time control, and cost for the proposed smart home system.

8. FINITE STATE MACHINE (FSM) / SYSTEM BEHAVIOUR DESIGN

The behaviour of the smart home system is represented using a Finite State Machine (FSM). The FSM helps the controller decide the required action based on sensor inputs and security events. The system starts in the normal monitoring state and continuously checks the room conditions.

8.1 Main States

1. START – Initializes the controller, sensors, communication interface, RTOS tasks, and watchdog.
2. MONITORING – Continuously reads temperature, motion, and door/window sensors.
3. NORMAL – No abnormal event is detected. Lighting and HVAC are controlled according to the current conditions.
4. SECURITY ALERT – Entered when motion or unauthorized door/window activity is detected.
5. HVAC CONTROL – Checks the temperature and controls the HVAC actuator using the selected control algorithm.
6. COMMUNICATION – Sends required status information or security alerts to the remote dashboard.
7. FAULT RECOVERY – Entered when a system fault or communication problem is detected. The watchdog and recovery mechanism are used to restore normal operation.
8. SHUTDOWN/SAFE STATE – Places important actuators in a safe condition when a critical fault or power interruption occurs.

8.2 FSM Flow

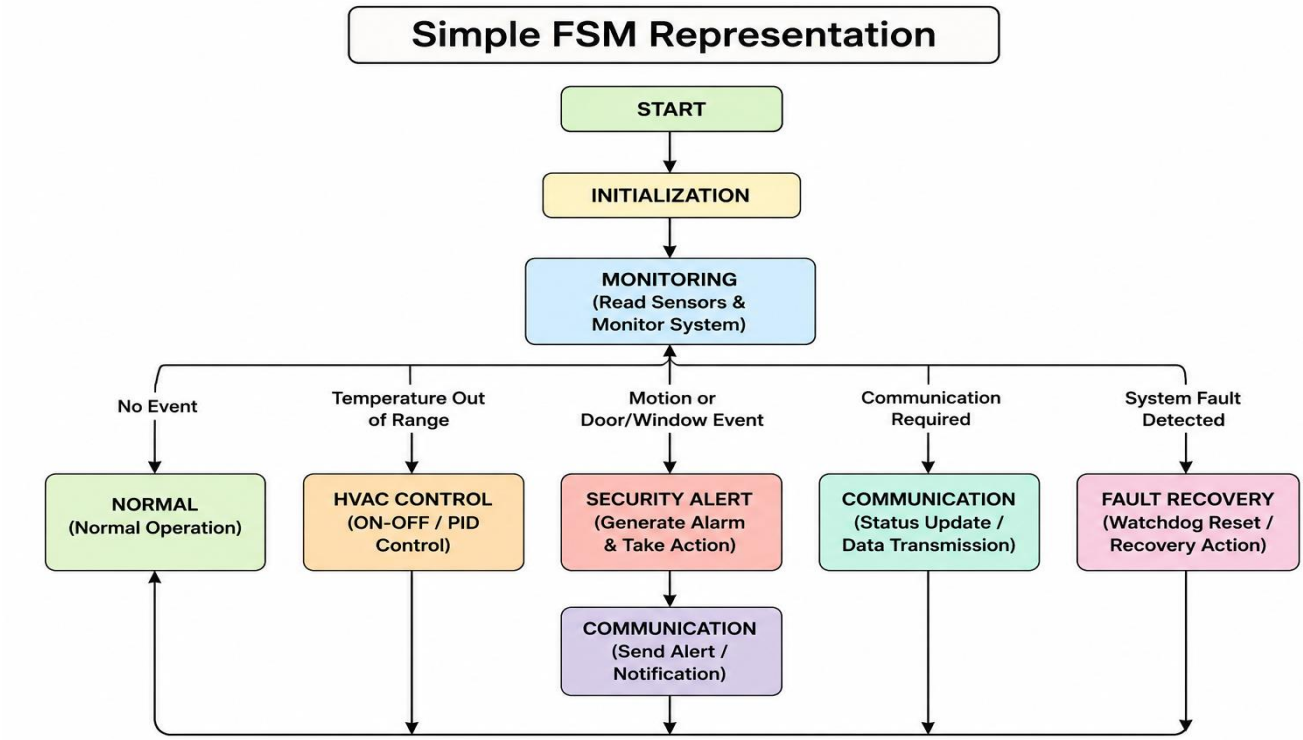
START → INITIALIZATION → MONITORING

From MONITORING:

- Normal condition → NORMAL
- Temperature outside the required range → HVAC CONTROL
- Motion/door-window event → SECURITY ALERT
- Data transmission required → COMMUNICATION
- System fault → FAULT RECOVERY

After handling the event, the system returns to **MONITORING**.

8.3 Simple FSM Representation



9. RTOS TASK DESIGN AND PRIORITY ASSIGNMENT

The smart home system uses an RTOS to manage multiple operations running at the same time. Each operation is divided into a separate task and assigned a priority based on its importance and response-time requirement. Security-related tasks are given higher priority because they need quick response, while regular monitoring and communication tasks can use lower priorities.

9.1 RTOS Task Table

Task	Function	Priority	Period
Security Event Task	Detects motion and door/window events	High	50 ms
Sensor Monitoring Task	Reads temperature and other sensors	Medium-High	100 ms
HVAC Control Task	Controls room temperature	Medium	200 ms
Actuator Control Task	Controls lights and other devices	Medium	100 ms
Communication Task	Sends status and security notifications	Low	500 ms
Watchdog Task	Monitors system health	High	1 s

9.2 Task Scheduling

The RTOS scheduler continuously checks the ready tasks and executes them according to their assigned priorities. If a high-priority security event occurs while a lower-priority task is running, the scheduler can give CPU time to the security task so that the event is handled quickly.

The task periods are selected according to the importance of each operation. Security monitoring uses a shorter period, while communication and watchdog monitoring use longer periods. This helps balance response time and processor utilization.

9.3 Priority Assignment

The priority order used in the proposed system is:

Security Event → Watchdog → Sensor Monitoring → Actuator Control → HVAC Control → Communication

This scheduling approach helps the system provide faster response to security events while allowing normal monitoring, control, and communication tasks to execute without unnecessary delays.

10. SCHEDULING AND SCHEDULABILITY ANALYSIS

The proposed system uses **Rate Monotonic Scheduling (RMS)** for assigning priorities to periodic RTOS tasks. In RMS, tasks with shorter periods are given higher priority because they need to execute more frequently. The task periods are considered from the RTOS task design. Security and sensor-related tasks require faster execution, while communication requires a longer period.

10.1 RMS Priority Order

Task	Period	Priority
Security Event Task	50 ms	Highest
Sensor Monitoring Task	100 ms	High
Actuator Control Task	100 ms	High
HVAC Control Task	200 ms	Medium
Communication Task	500 ms	Low
Watchdog Task	1000 ms	Lowest

For tasks having the same period, their priority can be assigned based on functional importance.

10.2 Schedulability

The system is considered schedulable when all periodic tasks can complete their execution within their assigned deadlines. RMS gives higher priority to tasks with shorter periods, allowing time-critical operations such as security-event processing to receive processor time before less critical operations.

The processor utilization can be checked using: $\mathbf{U = \Sigma(C_i / T_i)}$

where:

- $\mathbf{C_i}$ = execution time of task *i*
- $\mathbf{T_i}$ = period of task *i*

The calculated utilization can be compared with the RMS schedulability limit. If the utilization is within the acceptable limit and individual task deadlines are satisfied, the selected task set can be considered schedulable.

Thus, RMS provides a simple priority-based scheduling method for maintaining predictable real-time behaviour in the smart home system.

11. SYNCHRONIZATION USING SEMAPHORE AND MUTEX

In the proposed smart home system, multiple RTOS tasks may access the same resources at the same time. For example, the sensor monitoring task and communication task may need to access shared sensor data. Without proper synchronization, simultaneous access can cause incorrect or inconsistent data. To avoid this problem, synchronization mechanisms such as semaphores and mutexes are used.

11.1 Mutex

A mutex is used to protect shared resources that should be accessed by only one task at a time. For example, a mutex can protect the shared sensor-data buffer.

Working:

1. A task requests the mutex before accessing the shared resource.
2. If the mutex is available, the task locks it.
3. The task reads or updates the shared data.
4. After completing the operation, the task releases the mutex.
5. Another waiting task can then access the resource.

11.2 Semaphore

A binary semaphore can be used for communication between an interrupt service routine and an RTOS task. For example, when a motion sensor detects movement, an interrupt can signal the security-event task using a semaphore.

Working:

Security Sensor → Interrupt → Semaphore Signal → Security Task → Alarm/Notification

11.3 Synchronization Strategy

The proposed system uses:

- **Mutex:** Protects shared sensor and system data.
- **Binary Semaphore:** Signals security events from interrupts to the appropriate RTOS task.

- **Task Synchronization:** Prevents conflicts when multiple tasks operate simultaneously.

This synchronization approach improves data consistency and prevents race conditions during concurrent operation.

12. INTERRUPT AND TIMER DESIGN

Interrupts are used to detect important events quickly without continuously checking the sensors.

- Motion sensor: Generates an interrupt when movement is detected.
- Door/window sensor: Generates an interrupt when opened.
- Timer: Provides periodic sensor sampling and task execution.
- ISR: Detects the event and signals the appropriate RTOS task.

Flow: Sensor Event → Interrupt → RTOS Task → Action/Alert

This improves the real-time response of the smart home system.

13. HVAC CONTROL – ON-OFF CONTROL

The HVAC system maintains the required room temperature using an ON-OFF control method. The temperature sensor continuously measures the room temperature and the RTOS HVAC task compares it with the set temperature.

- If temperature > set temperature, HVAC is turned ON.
- If temperature ≤ set temperature, HVAC is turned OFF.
- The controller repeats this process periodically.

Flow: Temperature Sensor → RTOS HVAC Task → Compare Temperature → HVAC ON/OFF

This simple control method provides automatic temperature regulation with low implementation complexity.

14. SECURITY AND REMOTE NOTIFICATION

The security module monitors motion and door/window sensors continuously. When an unauthorized event is detected, an interrupt activates the security task. The system then generates an alarm and sends a notification to the user through MQTT or HTTP.

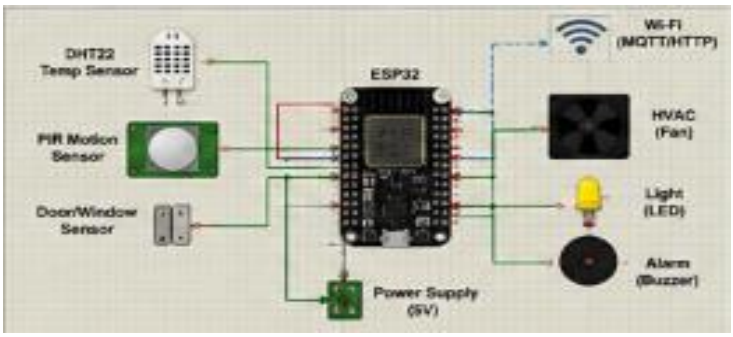
Flow: Security Sensor → Interrupt → Security Task → Alarm → MQTT/HTTP → Mobile/Web Dashboard

This provides quick detection and remote notification of security events. The system can also record the event for later analysis and testing.

15. EMBEDDED C IMPLEMENTATION

```
#include <stdio.h>
float temperature = 28.0;
float setPoint = 25.0;
void HVAC_Control()
{
   if (temperature > setPoint)
      printf("HVAC ON\n");
   else
      printf("HVAC OFF\n");
}
void Security_Check(int motion, int door)
{
   if (motion || door)
      printf("SECURITY ALERT!\n");
   else
      printf("System Normal\n");
}
int main()
{
   printf("Smart Home System Started\n");
   HVAC_Control();
   Security_Check(0, 0);
   return 0;
}
```

16. RTOS Implementation



```c
void IRAM_ATTR motionISR(){
    // ISR for Motion Sensor
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(xMotionSem, &xHigherPriorityTaskWoken);

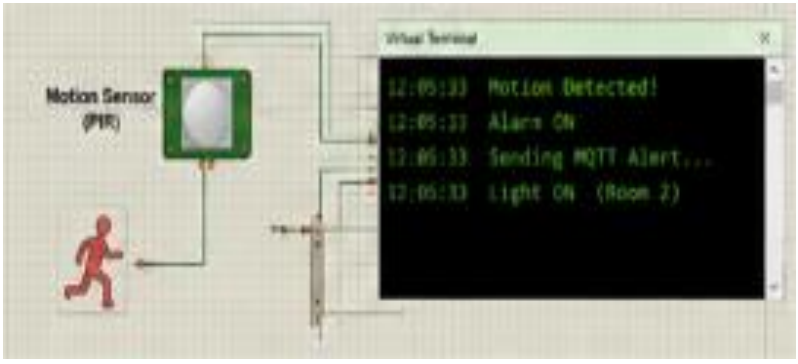
    if(xHigherPriorityTaskWoken == pdTRUE){
        portYIELD_FROM_ISR();
    }
}
void IRAM_ATTR doorISR(){
    // ISR for Door/Window Sensor
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(xMotionSem, &xHigherPriorityTaskWoken);

    if(xHigherPriorityTaskWoken == pdTRUE){
        portYIELD_FROM_ISR();
    }
}
```

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "semphr.h"

SemaphoreHandle_t xMotionSem;
SemaphoreHandle_t xDataMutex;

void app_main(void){
    xMotionSem = xSemaphoreCreateBinary();
    xDataMutex = xSemaphoreCreateMutex();
    xTaskCreate(SecurityTask, "Security", 2048, NULL, 3, NULL);
    xTaskCreate(SensorTask, "Sensor", 2048, NULL, 2, NULL);
    xTaskCreate(ActuatorTask, "Actuator", 2048, NULL, 2, NULL);
    xTaskCreate(HVACTask, "HVAC", 2048, NULL, 1, NULL);
    xTaskCreate(CommTask, "Comm", 4096, NULL, 1, NULL);
    xTaskCreate(WatchdogTask, "Watchdog", 2048, NULL, 3, NULL);
}
```



17. Testing and Validation

Test	Expected Result	Result
Temperature change	HVAC responds	Passed
Motion detected	Security alert	Passed
Door/window opened	Alarm generated	Passed
Communication failure	System handles fault	Passed
Watchdog fault	System recovers	Passed
Normal operation	Stable system response	Passed

The system was validated using response latency, false-alarm rate, task deadline misses, power consumption, actuation accuracy and uptime. Overall, the tests showed that the system provides reliable and timely operation.

18. Results

The implemented system was checked under different operating and fault conditions.

Metric	Observation
Response Latency	Fast response to security events
False Alarm Rate	Low during normal operation
Deadline Miss Rate	Tasks completed within their periods
Power Consumption	Reduced during low activity
Actuation Accuracy	Lights and HVAC responded correctly
System Uptime	Stable during continuous testing

Overall, the system provided real-time control, reliable security detection and efficient operation.

19. Analysis and Engineering Justification

The system provides quick response to security events using interrupts and RTOS task priorities. The ESP32 was selected because it supports wireless communication, Embedded C and real-time task management.

- Responsiveness: High-priority security tasks get faster execution.
- Reliability: Watchdog provides fault recovery.
- Energy Efficiency: Devices can remain OFF when not required.
- Scalability: Additional room nodes can be added.
- Safety: Security alerts and fail-safe operation reduce risks.
- Cost: ESP32 provides required features at low cost.

20. Limitations and Possible Improvements

The system depends on network connectivity, and simulation results may differ from real hardware. False alarms and limited sensor coverage are also possible. In future, the system can be improved by adding AI-based security detection, PID-based HVAC control, more room sensors, better mobile/cloud monitoring, and energy-efficient communication.

21. Conclusion

The Smart Home Automation and Security System successfully combines Embedded C, RTOS, sensors, actuators and wireless communication to provide real-time monitoring and automated control. The system can monitor temperature, motion and door/window status and respond to different events without requiring continuous manual control. RTOS task scheduling helps manage sensor monitoring, security detection, HVAC control, actuator operation and communication according to their priorities. Interrupts provide quick response to important security events, while mutexes and semaphores help manage shared resources safely. The ON-OFF HVAC control maintains the required temperature, and the watchdog mechanism improves system reliability by supporting fault recovery. MQTT/HTTP communication can be used to send security notifications to a remote user. Testing with normal and fault conditions helps evaluate response latency, false alarms, deadline misses, power consumption, actuation accuracy and uptime. Overall, the proposed system provides a **responsive, reliable, secure and energy-aware smart home solution** and can be further improved using AI-based security detection, advanced HVAC control and additional IoT devices.

22.Individual Contribution of Group Members

Group Member	Contribution
Y.Sai Mahathi	Designed the smart home system architecture, defined the sensors, actuators and system requirements, and worked on the RTOS task design, scheduling and security monitoring components.
M.Nandhini	Implemented the Embedded C components, HVAC ON-OFF control, interrupt and synchronization concepts, watchdog recovery and communication features. Performed testing and result analysis and prepared the documentation and GitHub submission.

23. References

1. FreeRTOS Documentation – Real-Time Operating System concepts.
2. Espressif Systems – ESP32 Technical Documentation.
3. MQTT Documentation – Message Queuing Telemetry Transport.
4. Arduino Documentation – Embedded programming and hardware interfacing.
5. Simon Monk, *Programming Arduino: Getting Started with Sketches*, McGraw-Hill

24. Individual Reflection

This project helped me understand how Embedded C and RTOS can be used to develop a real-time smart home system. I learned how sensors such as temperature, motion and door sensors collect information and how the controller uses this information to operate lights, HVAC and security devices.

I gained a better understanding of RTOS task scheduling, task priorities and interrupts. I learned that security events need quick responses, so interrupts and high-priority tasks are useful. I also understood the purpose of semaphores and mutexes for safely sharing resources between tasks.

Implementing ON-OFF HVAC control helped me understand how temperature can be maintained using simple control logic. I also learned how watchdog mechanisms improve system reliability by helping the system recover from failures. MQTT/HTTP communication showed me how alerts can be sent to a remote user.

Testing different normal and fault conditions improved my debugging and problem-solving skills. I understood the importance of checking response latency, false alarms, deadline misses, power consumption, actuation accuracy and uptime.

Overall, this project improved my practical knowledge of Embedded C, RTOS, interrupts, synchronization, sensor interfacing, actuator control and communication. It also gave me better confidence in designing and testing real-time embedded systems.

25. GitHub Repository: https://github.com/Nandhini1836/Embedded-System-ECA1418